

```
# =====  
# Roll no : 60    PRN: 0124UITM1060  
# Name : Kshitij Shinde  
# Dept. : Information Technology (Third Year)  
# =====  
# Assignment 3:  
# Credit Risk Assessment using Random Forest Classifier  
# =====
```

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import (  
    confusion_matrix,  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    classification_report,  
    roc_curve,  
    roc_auc_score  
)
```

```
# -----
```

```
# STEP 1: LOAD DATASET
```

```
# -----
```

```
df = pd.read_csv("loan_data.csv")
```

```
print("First 5 rows:")
```

```
print(df.head())
```

```
print("\nDataset information:")
```

```
print(df.info())
```

```
print("\nMissing values:")
```

```
print(df.isnull().sum())
```

```
print("\nTarget distribution:")
```

```
print(df["default"].value_counts())
```

```
# -----
```

```
# STEP 2: SELECT FEATURES AND TARGET
```

```
# -----
```

```
features = [
```

```
    "Age",
```

```
    "Income",
```

```
    "Loan Amount",
```

```
    "Credit Score",
```

```
    "Employment Duration",
```

"Debt-to-Income Ratio"

]

X = df[features]

y = df["default"]

STEP 3: HANDLE MISSING VALUES

X = X.fillna(X.median())

STEP 4: TRAIN-TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split(

X,

y,

test_size=0.20,

random_state=42,

stratify=y

)

STEP 5: FEATURE SCALING

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# -----
```

```
# STEP 6: BUILD RANDOM FOREST MODEL
```

```
# -----
```

```
model = RandomForestClassifier(
```

```
    n_estimators=100,
```

```
    random_state=42
```

```
)
```

```
# -----
```

```
# STEP 7: TRAIN MODEL
```

```
# -----
```

```
model.fit(X_train, y_train)
```

```
# -----
```

```
# STEP 8: MAKE PREDICTIONS
```

```
# -----
```

```
y_pred = model.predict(X_test)
```

```
# -----
```

```
# STEP 9: CONFUSION MATRIX
```

```
# -----
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    cmap="Blues",  
    xticklabels=["No Default", "Default"],  
    yticklabels=["No Default", "Default"]  
)
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

```
# -----
```

```
# STEP 10: ACCURACY
```

```
# -----
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nAccuracy:", accuracy)
```

```
# -----
```

```
# STEP 11: PRECISION
```

```
# -----
```

```
precision = precision_score(
```

```
    y_test,
```

```
    y_pred,
```

```
    zero_division=0
```

```
)
```

```
print("Precision:", precision)
```

```
# -----
```

```
# STEP 12: RECALL
```

```
# -----
```

```
recall = recall_score(
```

```
    y_test,
```

```
    y_pred,
```

```
    zero_division=0
```

```
)
```

```
print("Recall:", recall)
```

```
# -----
```

```
# STEP 13: F1-SCORE
```

```
# -----
```

```
f1 = f1_score(
```

```
    y_test,
```

```
    y_pred,
```

```
    zero_division=0
```

```
)
```

```
print("F1-Score:", f1)
```

```
# -----
```

```
# STEP 14: CLASSIFICATION REPORT
```

```
# -----
```

```
print("\nClassification Report:")
```

```
print(
```

```
    classification_report(
```

```
        y_test,
```

```
        y_pred,
```

```
        zero_division=0
```

```
    )
```

```
)
```

```
# -----
```

```
# STEP 15: ROC CURVE AND AUC
```

```
# -----
```

```
y_prob = model.predict_proba(X_test)[:, 1]
```

```
fpr, tpr, thresholds = roc_curve(  
    y_test,  
    y_prob  
)
```

```
auc = roc_auc_score(  
    y_test,  
    y_prob  
)
```

```
print("ROC-AUC Score:", auc)
```

```
# Plot ROC Curve
```

```
plt.figure(figsize=(7, 5))
```

```
plt.plot(  
    fpr,  
    tpr,  
    label=f"AUC = {auc:.2f}"  
)
```

```
plt.plot(  
    [0, 1],  
    [0, 1],  
    linestyle="--")
```



```
)
```

```
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curve")
```

```
plt.legend()
```

```
plt.show()
```

```
# -----
```

```
# STEP 16: FEATURE IMPORTANCE
```

```
# -----
```

```
importance = pd.Series(  
    model.feature_importances_,  
    index=features  
)  
.sort_values(ascending=False)
```

```
print("\nFeature Importance:")
```

```
print(importance)
```

```
plt.figure(figsize=(8, 5))
```

```
importance.plot(  
    kind="bar"  
)
```

```
plt.xlabel("Features")
```

```
plt.ylabel("Importance")
```

```
plt.title("Random Forest Feature Importance")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```